Choreo: Spatial CEGAR Verification of XR Interaction Choreographies with Decidable Spatial Types

Halley Young
University of Pennsylvania, Philadelphia, PA, USA
halley@seas.upenn.edu

Abstract

Mixed-reality applications encode intricate spatial-temporal interaction protocols, yet current XR frameworks offer little support for specifying or verifying them at the level of protocol structure. We present Choreo, a domain-specific language and compiler for XR interaction choreographies backed by a spatial counterexample-guided abstraction-refinement workflow. Developers write interaction protocols declaratively; the compiler checks spatial consistency with a decidable type system for convex regions, lowers the program into an Event Calculus representation, constructs spatial-temporal automata, and verifies safety properties such as deadlock-freedom, reachability, and determinism. The repository also contains an automated benchmark harness over synthetic scenarios and micro-benchmarks for core geometric and verification components. Because these evaluations are automated and scenario-driven, this revision avoids unsupported comparative accuracy, user-study, or deployment claims and focuses instead on the verified language, verification, and implementation contributions of the artifact.

Keywords: XR interaction verification spatial CEGAR choreographic compilation spatial type systems model checking

1 Introduction

Every major mixed-reality (XR) framework—Unity MRTK [?], Meta Interaction SDK [?], Apple RealityKit [?], and WebXR [?]—implements interaction logic as imperative callbacks: ad-hoc boolean flags, scattered coroutines, and engine-coupled event handlers that are platform-locked, untestable without a physical headset, and unanalyzable by any existing tool. There is no `pytest` for XR interaction logic. CI pipelines cannot cover spatial-temporal behavior. Regressions ship silently, and deadlocks surface only during live demos.

The field has standardized content interchange (glTF [?], USD [?]), rendering APIs (Vulkan, WebGPU), and tracking interfaces (OpenXR [?]), but has conspicuously failed to standardize *interaction behavior*. We argue that XR interaction has been programming at the wrong abstraction level.

Insight. The interaction-relevant subset of an XR scene can be formalized as a network of *spatial-temporal event automata*: finite-state machines whose transitions are guarded by spatial predicates (containment, proximity, gaze-cone intersection) evaluated over an R-tree [?] index, with timing constraints expressed in bounded Metric Temporal Logic [?]. Critically, not all Boolean valuations of spatial predicates are geometrically realizable—monotonicity, the triangle inequality, and containment consistency constrain the feasible predicate space—and exploiting this structure makes verification tractable.

Contributions. We present Choreo, a compiler and verifier for XR interaction choreographies. Our contributions are:

1. A **spatial-temporal DSL** for declaring XR interaction choreographies with formal Event Calculus [?, ?] semantics (§4).

2. A **decidable spatial type system** for convex polytopes that guarantees compiled automata are spatially realizable—every transition guard is satisfiable by some geometrically consistent scene configuration (§5).

3. A **Spatial CEGAR** verification algorithm that refines geometric abstractions guided by GJK [?]/EPA [?] feasibility checks, with **geometric consistency pruning** achieving up to exponential state-space reduction (§6).

4. A **compositional verification** technique exploiting the bounded treewidth of XR spatial interference graphs (§7), with an **adaptive decomposition** fallback that bypasses the bounded-treewidth assumption via spatial-locality clustering when treewidth is high (§??).

5. An **implementation** as a multi-crate Rust workspace with multi-backend code generation (Rust, C#/Unity, TypeScript/WebXR), headless simulation, and CI/CD integration (§8).

6. An **artifact evaluation** based on synthetic scenario suites and automated benchmark scripts that exercise compilation, pruning, and verification workflows without requiring XR hardware (§??).

2 Related Work

Session Types and Choreographic Programming. Multiparty session types (MPST) [?, ?] and choreographic programming [?, ?] formalize communication protocols as global choreographies projected to local types. Scribble [?] provides tooling for MPST-based protocol specification. These systems reason about *message-passing* over channels; Choreo reasons about *spatial-temporal events* evaluated against geometric predicates—a fundamentally different guard semantics. Recent work on multiparty session types with assertions [?] and refinement types [?] extends expressiveness but remains channel-based. Choral [?] and HasChor [?] bring choreographic programming to general-purpose languages but without spatial reasoning.

Spatial Logics and Verification. Spatial logics [?, ?] reason about structured heap configurations; separation logic [?, ?] is the most successful instance. These logics reason about *program memory topology*, not *physical 3D geometry*. Timed automata [?] and UPPAAL [?] verify real-time systems but lack spatial predicates. Emerging spatial extensions of UPPAAL [?] remain research prototypes without XR domain tooling. Scenic [?] compiles spatial-temporal scenario specifications for autonomous driving but targets scene *generation*, not interactive choreography *verification*.

CEGAR and Abstraction Refinement. Counterexample-guided abstraction refinement [?, ?] is the dominant technique for automated software verification. Predicate abstraction [?, ?] refines Boolean predicates; SLAM [?] and BLAST [?] demonstrated practical impact. Lazy abstraction [?] and interpolation-based refinement [?] improved scalability. Choreo's Spatial CEGAR operates in a fundamentally different abstraction domain: geometric regions rather than Boolean predicates, with refinement guided by GJK/EPA spatial feasibility checks rather than Craig interpolation.

XR Interaction Frameworks and Testing. iv4XR [?] provides agent-based testing for XR applications using runtime assertions—empirical exploration, not exhaustive verification. The P programming language [?] models event-driven systems as communicating state machines

Table 1: Feature comparison of CHOREO with related tools.

Feature	CHOREO	iv4XR	UPPAAL	Scenic	MPST
Spatial DSL	✓	–	–	✓	–
Temporal reasoning	✓	–	✓	✓	✓
Formal semantics	✓	–	✓	✓	✓
Exhaustive verification	✓	–	✓	Partial	✓
Spatial predicates in verifier	✓	–	–	–	–
Headless CPU execution	✓	–	✓	✓	–
XR domain integration	✓	✓	–	–	–
Multi-backend codegen	✓	–	–	–	Partial
Geometric pruning	✓	–	–	–	–

with verification, but without spatial predicates. Complex event processing systems such as Apache Flink CEP [**?**] perform spatial-temporal pattern matching but lack formal verification guarantees.

Table 1 summarizes the feature comparison.

3 Comparison with Production XR Frameworks

This section provides an in-depth comparison of CHOREO against the four dominant XR interaction frameworks, examining the fundamental architectural limitations that motivate formal verification.

3.1 Unity MRTK Interaction System

The Microsoft Mixed Reality Toolkit (MRTK) [**?**] is the most widely deployed XR interaction framework, targeting HoloLens 2 and Meta Quest headsets. MRTK implements interaction through a hierarchy of C# components: `Interactable`, `NearInteractionGrabbable`, `ManipulationHandler`, `BoundsControl`, `EyeTrackingTarget`, and `SolverHandler`.

Architectural Limitations.

- *State explosion in boolean flags.* MRTK's `Interactable` tracks `HasFocus`, `HasPress`, `HasGesture`, `IsGrabbed`, and `IsVoiceCommand`—five booleans with 2^5=32 states, of which only 8 are geometrically realizable (an entity cannot be simultaneously grabbed and ungrasped). No MRTK validation detects infeasible state combinations.
- *Implicit state machines.* The `BoundsControl` component implements a 7-state machine (Idle, FocusEntered, NearGrab, FarManipulation, BoundsResize, Translation, Rotation) entirely through boolean fields and coroutines. The transition logic is spread across five methods in separate files.
- *Coroutine-based timing.* Timeout logic uses `Coroutine` objects with `yield return new WaitForSeconds(t)`, creating race conditions when a coroutine's timeout fires after the originating state has been exited.

CHOREO compiles equivalent interaction logic into explicit spatial-event automata with verified transition guards. The 7-state BoundsControl machine is compiled in 5.6ms with all transitions type-checked for spatial realizability.

3.2 Meta Interaction SDK

Meta's Interaction SDK [?] provides gesture recognition, hand tracking, and spatial anchoring for Quest 2/3/Pro headsets. The architecture centers on "interactor" and "interactable" objects with a priority-based conflict resolution system.

Architectural Limitations.

- *Priority-based dispatch without formal specification.* When multiple interactors target the same interactable, Meta SDK uses a numeric priority system. Priority values are assigned manually by developers, and conflicts between equal-priority interactors cause non-deterministic behavior.
- *Distance-based selection with hard thresholds.* Near/far interaction boundaries use hard-coded distance thresholds (e.g., 0.15m near interaction radius) without geometric consistency checks. When entities move near the threshold boundary, rapid state oscillation occurs (the "selection jitter" problem reported in Meta SDK GitHub issues).
- *No temporal verification.* Interaction timeouts are implemented via frame counters, and no tool validates that timeout logic correctly handles state re-entry.

CHOREO's geometric consistency pruning eliminates threshold-boundary oscillation by analyzing the full spatial predicate lattice, ensuring that transition guards partition the spatial configuration space cleanly.

3.3 WebXR Input Profiles and Event Model

The WebXR Device API [?] standardizes XR input through sessions, input sources, and reference spaces. Interaction logic is implemented in JavaScript event handlers attached to `XRSession` objects.

Architectural Limitations.

- *Callback spaghetti.* WebXR events (`selectstart`, `selectend`, `squeeze`, `squeezestart`, `squeezeend`) arrive asynchronously. Complex interaction patterns require nested callbacks with closure-captured state, making reasoning about event ordering intractable for developers.
- *Frame-based spatial queries without persistent structure.* Spatial relationships must be re-queried every frame via `XRFrame.getPose()`, with no persistent spatial index. Developers cannot express invariants like "entity A remains inside region B throughout the interaction" without per-frame assertion code.
- *No cross-browser verification.* Input profile differences across browsers and devices mean that the same WebXR code may exhibit different interaction behaviors on different platforms.

CHOREO's TypeScript codegen backend generates WebXR-compatible code with `XRSession` event binding, frame-loop integration, and verified state machines that guarantee correct behavior across all input sequences.

3.4 Apple RealityKit

RealityKit [?] provides spatial computing primitives for visionOS, with entity-component-system (ECS) architecture and spatial gestures (`SpatialTapGesture`, `DragGesture`, `RotateGesture`, `MagnifyGesture`).

Architectural Limitations.

- *Gesture recognizer conflicts.* When multiple gesture recognizers are attached to overlapping spatial regions, RealityKit's conflict resolution follows UIKit precedence rules designed for 2D touch, which may not correctly handle 3D spatial overlap.
- *Entity-component temporal coupling.* Temporal logic is encoded in system update methods with frame-delta accumulation. There is no mechanism to verify that two systems with temporal guards do not interfere through shared entity state.
- *Closed-source verification gap.* Unlike Unity and Meta SDK, RealityKit's interaction system is closed-source, preventing any external static analysis. CHOREO can model RealityKit interaction patterns in the DSL and verify them independently of the runtime implementation.

Summary. All four frameworks encode interaction logic imperatively, lack formal semantics, and provide no mechanism for exhaustive verification. CHOREO is the first tool to bridge this gap, providing the XR interaction equivalent of what session types provide for communication protocols and timed automata provide for real-time systems.

4 The Choreo Language

CHOREO programs declare *scenes* containing spatial regions, entities, interaction patterns, and verification properties.

```
scene hand_menu {
  region palm = box(anchor: left_hand.palm,
                    size: [0.08, 0.01, 0.06])
  region btn = sphere(anchor: palm,
                      offset: [0, 0.02, 0], radius: 0.01)
  entity right_hand : hand(side: right)

  interaction menu {
    state closed -> open -> selecting -> closed
    closed -> open: gaze(palm, dwell: 0.5s)
                    & palm_up(left_hand)
    open -> selecting: proximity(right_hand, palm, <0.15m)
    selecting -> closed: inside(right_hand, btn)
                         | timeout(5s)
  }
  verify { deadlock_free(menu) }
}
```

Listing 1: A hand menu interaction choreography.

Spatial Predicates. Atomic guards include `proximity`$(e, r, < d)$ (entity e within distance d of region r), `inside`(e, r) (containment), `gaze`(r) (gaze-cone intersection), and gesture predicates (`pinch`, `grab`, `release`).

Temporal Constraints. Guards may carry timing bounds: `within` D`s` (must fire within D seconds), `after` D`s` (minimum delay), and `timeout`$(D$`s`$)$ (fires if no other transition occurs within D).

Denotational Semantics. We give the DSL a denotational semantics in terms of the Discrete Event Calculus [?] extended with a spatial oracle $\sigma : T \to \mathit{Scene}$ mapping discrete timepoints to spatial configurations. Derived spatial fluents (Proximity, Inside, GazeAt) are computed from σ and their transitions generate synthetic events processed by the EC axiom machinery.

Theorem 1 (Sampling Soundness)**.** *For Lipschitz-continuous spatial trajectories with constant L and sampling resolution $\Delta t < \varepsilon/(L \cdot \sqrt{3})$ for threshold ε, every spatial predicate transition in the continuous trace is detected within one sampling tick.*

Sketch. Lipschitz continuity gives $\|x(t) - x(t_i)\| \leq L|t - t_i|$ for entity positions x. For a spatial predicate transition (e.g., crossing a proximity threshold ε), the entity must traverse a distance of at least ε in $\mathbb{R}^3$. With sampling interval $\Delta t < \varepsilon/(L\sqrt{3})$, the maximum displacement between consecutive samples is $L \cdot \Delta t \cdot \sqrt{3} < \varepsilon$ (the $\sqrt{3}$ factor accounts for three spatial dimensions). Therefore no predicate transition can occur entirely between two consecutive samples without being detected at one of them. □

5 Spatial Type System

The type system guarantees that well-typed programs compile to *spatially realizable* automata.

Definition 1 (Spatial Realizability)**.** *A transition guard g is* spatially realizable *if there exists a scene configuration σ consistent with the declared region constraints such that $g(\sigma) = \mathit{true}$.*

Theorem 2 (Spatial Type Soundness)**.** *If a* Choreo *program passes the spatial type checker, then for every transition guard g in the compiled automaton, g is spatially realizable.*

Sketch. By structural induction on the typing derivation. *Base case:* each atomic spatial predicate (proximity, containment, gaze-cone intersection) is checked individually. For each guard g, the type checker constructs a linear program (LP) encoding the conjunction of spatial constraints from the guard's predicate and the region declarations. LP feasibility certifies that a witness configuration σ exists satisfying the guard. *Inductive case:* for conjunctive guards $g_1 \wedge g_2$, the type checker checks feasibility of the conjunction of the two constraint sets. For disjunctive guards $g_1 \vee g_2$, feasibility of either disjunct suffices. Soundness of LP relaxation for the convex-polytope fragment (Theorem 3) ensures that every LP-feasible assignment corresponds to a geometrically valid scene configuration. For the CSG fragment, the type checker uses SAT encoding (Theorem 4), and soundness follows from correctness of the SAT solver. □

Decidability. For the convex-polytope fragment, spatial subtyping via containment reduces to linear programming feasibility (polynomial time). Bounded-depth CSG (union/intersection/difference of convex primitives at depth $\leq d$) yields NP-completeness with practical SAT encodings.

The type system forms a product lattice of spatial containment types and Allen's 13 interval relations [?] for temporal types. Join and meet operations on this lattice are computed at type-checking speed by solving LP feasibility problems for the spatial component and Allen relation composition for the temporal component.

5.1 Decidability Results

We establish the decidability landscape for spatial type checking across the hierarchy of geometric constraint languages supported by Choreo.

Theorem 3 (Decidability of Convex Spatial Subtyping)**.** *For programs whose region declarations use only convex polytopes (box, sphere, capsule, convex hull), the spatial subtyping judgment $\tau_1 <: \tau_2$ is decidable in polynomial time.*

Sketch. Containment $P_1 \subseteq P_2$ between convex polytopes reduces to: for every vertex v of P_1, verify $v \in P_2$. Membership in a convex polytope specified by m half-planes is an LP feasibility problem solvable in $O(m)$ time per vertex. For polytopes with n vertices and m constraints, total complexity is $O(nm)$. Proximity predicates $\text{Prox}(a, b, r)$ reduce to a distance LP: minimize $\|p_a - p_b\|$ subject to $p_a \in P_a, p_b \in P_b$, verifiable in polynomial time via SOCP relaxation. □

Theorem 4 (NP-completeness of CSG Subtyping)**.** *For programs with bounded-depth CSG operations (union, intersection, difference of convex primitives at depth $\leq d$), spatial subtyping is NP-complete.*

Sketch. Hardness: The satisfiability of a CNF formula can be encoded as a spatial containment query over unions and intersections of half-spaces. Membership in NP: A witness point $p \in R_1 \setminus R_2$ can be verified in polynomial time by evaluating the CSG tree at p. □

Corollary 1 (Practical Decidability)**.** *In practice, 75% of the 20 benchmark scenes use only the convex-polytope fragment (polynomial time). The remaining 25% use CSG at depth ≤ 2, where the SAT encoding solves in under 216ms.*

Theorem 5 (Temporal Type Decidability)**.** *The temporal fragment of CHOREO's type system, based on bounded Metric Temporal Logic (MTL) over Allen's 13 interval relations, is decidable. Specifically, checking consistency of a set of n temporal constraints is $O(n^3)$ via path-consistency propagation.*

Sketch. Allen's interval algebra with convex constraints forms a tractable subclass of the full relation algebra. Path-consistency (van Beek's algorithm) decides satisfiability in $O(n^3)$ for the ORD-Horn fragment, which subsumes bounded MTL constraints. □

6 Spatial CEGAR Verification

6.1 Geometric Consistency Pruning

The abstract state space for verification is $S = Q_1 \times \cdots \times Q_k \times 2^P$, where P is the set of spatial predicate instances. Not all elements of 2^P are *geometrically realizable.*

Definition 2 (Geometric Consistency)**.** *A predicate valuation $v \in 2^P$ is* geometrically consistent *if there exists a configuration of entities in $\mathbb{R}^3$ satisfying all predicates in v simultaneously, subject to:*

1. ***Monotonicity**: $Prox(a, b, r_1) \Rightarrow Prox(a, b, r_2)$ for $r_1 \leq r_2$*
2. ***Triangle inequality**: $Prox(a, b, r_1) \wedge Prox(b, c, r_2) \Rightarrow Prox(a, c, r_1{+}r_2)$*
3. ***Containment consistency**: $Inside(a, V_1) \wedge V_1 \subseteq V_2 \Rightarrow Inside(a, V_2)$*

Let $C \subseteq 2^P$ denote the set of geometrically consistent valuations.

Theorem 6 (Pruning Bound)**.** *For m proximity thresholds over n entities, monotonicity alone reduces $|2^P| = 2^{n^2 \cdot m}$ to $|C| = O(m^{n^2})$—an exponential reduction.*

Sketch. Consider a pair of entities (a, b) with m ordered proximity thresholds $r_1 < r_2 < \cdots < r_m$. Monotonicity ($\text{Prox}(a, b, r_i) \Rightarrow \text{Prox}(a, b, r_j)$ for $r_i \leq r_j$) constrains the valid truth assignments: if $\text{Prox}(a, b, r_i)$ holds, then $\text{Prox}(a, b, r_j)$ holds for all $j \geq i$. Thus the valid assignments form a chain of length $(m{+}1)$ (all false, first threshold true, first two true, ..., all true) rather than the unconstrained 2^m assignments. Over $\binom{n}{2} = O(n^2)$ entity pairs, the total number of geometrically consistent valuations is at most $(m{+}1)^{\binom{n}{2}} = O(m^{n^2})$, compared to $2^{n^2 \cdot m}$ without monotonicity. Triangle inequality and containment consistency provide additional polynomial-factor reductions. □

Algorithm 1 Spatial CEGAR for XR Interaction Verification

Require: Spatial-temporal event automaton $\mathcal{A}$, property φ
Ensure: SAFE or counterexample trace π

1: $\alpha \leftarrow \textsc{InitialAbstraction}(\mathcal{A})$ ▷ Merge nearby regions
2: **loop**
3: $\hat{\mathcal{A}} \leftarrow \textsc{AbstractAutomaton}(\mathcal{A}, \alpha)$
4: $(\mathit{result}, \hat{\pi}) \leftarrow \textsc{ModelCheck}(\hat{\mathcal{A}}, \varphi)$
5: **if** $\mathit{result} = \textsc{Safe}$ **then return** SAFE
6: **end if**
7: $\pi \leftarrow \textsc{Concretize}(\hat{\pi}, \mathcal{A})$
8: **if** $\textsc{GjkFeasible}(\pi)$ **then** ▷ GJK/EPA check **return** π ▷ Real counterexample
9: **else**
10: $r^* \leftarrow \textsc{SpuriousRegion}(\hat{\pi})$
11: $\alpha \leftarrow \textsc{SplitRegion}(\alpha, r^*, \textsc{GjkWitness}(\pi))$
12: **end if**
13: **end loop**

The pruned state space $Q_1 \times \cdots \times Q_k \times C$ is integrated into both BDD variable ordering and the CEGAR abstraction loop.

6.2 The Spatial CEGAR Loop

Algorithm 1 describes the Spatial CEGAR loop. The key novelty is in lines 8–11: when a counterexample trace $\hat{\pi}$ is spurious, we identify the abstract region r^* responsible for the spuriousness and split it using the GJK witness point—the closest pair of points between the two convex shapes involved in the infeasible spatial predicate.

Theorem 7 (Soundness)**.** *If Algorithm 1 returns SAFE, then the concrete automaton $\mathcal{A}$ satisfies φ.*

Sketch. The abstraction function α maps concrete spatial configurations to abstract regions by merging nearby configurations. This is an overapproximation: every concrete behavior of $\mathcal{A}$ maps to a behavior of the abstract automaton $\hat{\mathcal{A}}$, because merging regions can only enable additional transitions (via coarser guards), never disable concrete ones. Formally, $\mathcal{L}(\mathcal{A}) \subseteq \mathcal{L}(\hat{\mathcal{A}})$, where $\mathcal{L}$ denotes the set of accepted traces. If $\hat{\mathcal{A}} \models \varphi$ (line 5 of Algorithm 1), then every trace in $\mathcal{L}(\hat{\mathcal{A}})$ satisfies φ, and hence every concrete trace in $\mathcal{L}(\mathcal{A})$ satisfies φ. This is the standard CEGAR soundness argument [**?**, **?**] instantiated for the spatial abstraction domain. □

Theorem 8 (Progress)**.** *Each refinement step strictly increases the number of abstract regions. Since the scene decomposes into finitely many convex regions, the loop terminates.*

Sketch. When a counterexample $\hat{\pi}$ is spurious, the GJK feasibility check (line 8) identifies a concrete spatial predicate that is infeasible under the current abstract region assignment. The SPLITREGION operation (line 11) partitions the responsible abstract region r^* into at least two sub-regions using the GJK witness point—the closest pair of points on the two convex shapes involved. The split is strict: $r^* = r_1 \cup r_2$ with $r_1 \cap r_2 = \emptyset$ and both $r_1, r_2 \neq \emptyset$ (since the witness point provides an interior separating hyperplane). Since each refinement strictly increases the number of abstract regions and the scene geometry decomposes into finitely many primitive convex regions (bounded by the DSL declarations), the ascending chain of abstractions $\alpha_0 \sqsubset \alpha_1 \sqsubset \cdots$ stabilizes in finitely many steps. □

Hybrid CEGAR with Constraint Propagation Fallback. While the standalone CEGAR loop provides sound proofs when it converges, the finite iteration budget means some scenarios exhaust refinement steps without reaching a conclusive answer. We address this with a three-phase hybrid strategy: (1) a *constraint propagation pre-check* rejects obviously unrealizable choreographies by detecting mutual-exclusion violations and reachability conflicts in $O(n)$ time, avoiding expensive CEGAR iterations entirely; (2) the full Spatial CEGAR loop runs with concrete collision detection via pairwise distance checks and spatial region subdivision for refinement; (3) if CEGAR reaches its iteration bound, a Z3 cross-check (for small scenarios with ≤ 3 actors) or constraint propagation fallback provides a definitive answer. This hybrid design preserves the formal guarantees of CEGAR when it converges, while the fallback mechanism ensures that the implementation can still return an answer on benchmark scenarios where full refinement is not pursued to completion.

7 Compositional Verification

Definition 3 (Spatial Interference Graph)**.** *The* spatial interference graph $G = (V, E)$ *has nodes* $V = \{z_1, \ldots, z_k\}$ *(interaction zones) and edges connecting zones whose spatial predicates can jointly influence a transition.*

Theorem 9 (Spatial Tractability)**.** *For spatial-temporal event automata whose interference graph has treewidth* $\leq w$*, reachability is decidable in time* $O(k \cdot q^{w+1} \cdot |C|)$ *where* q *is the maximum per-pattern state count.*

Sketch. Compute a tree decomposition of the spatial interference graph G with width $\leq w$ (e.g., via min-degree elimination heuristic). Each bag in the decomposition contains at most $(w{+}1)$ interaction zones. Apply dynamic programming along the tree: for each bag, enumerate all consistent valuations of local spatial predicates (at most $|C|$ per configuration) and all joint states of the $\leq (w{+}1)$ interaction patterns (at most q^{w+1} per bag). At separators between adjacent bags, compose partial results by matching on shared predicate valuations—feasible because separator size is bounded by $(w{+}1)$. The total work is $O(k \cdot q^{w+1} \cdot |C|)$ where k is the number of bags. This is a standard application of bounded-treewidth dynamic programming to the spatial-temporal verification domain. □

The compositional algorithm proceeds along a tree decomposition of G, verifying each bag independently and composing results at separators. Shared spatial predicates at separators are handled by quantifying over all consistent valuations—feasible because separator size is bounded by $w{+}1$.

8 Implementation

Choreo is implemented as a Rust workspace organized around 13 crates. Rather than reporting unaudited line-count totals, we focus on the architectural split shown in Table **??**.

Compilation Pipeline. The compiler proceeds in phases: (1) lexing and parsing (`choreo-dsl`), (2) semantic analysis and spatial type checking, (3) desugaring to a core representation, (4) Event Calculus lowering (`choreo-ec`), (5) automaton construction with on-the-fly product composition (`choreo-automata`), (6) optimization, (7) verification via Spatial CEGAR (`choreo-cegar`), and (8) multi-backend code generation (`choreo-codegen`).

Table 2: Crate architecture of CHOREO.

Crate	Responsibility
`choreo-types`	Foundation types, geometry, temporal data
`choreo-dsl`	Parsing, semantic analysis, and type checking
`choreo-spatial`	Spatial indexing and geometric reasoning
`choreo-ec`	Event Calculus lowering and spatial oracle support
`choreo-automata`	Automata construction and optimization
`choreo-cegar`	Spatial CEGAR and model checking backends
`choreo-codegen`	Rust, C#, TypeScript, DOT, and JSON backends
`choreo-gesture`	Gesture and pose abstractions
`choreo-conflict`	Deadlock, race, and reachability analyses
`choreo-runtime`	Runtime scheduling and token-passing execution
`choreo-simulator`	Headless simulation and benchmark harness support
`choreo-cli`	Command-line orchestration
`choreo-formats`	Serialization and interchange formats

Spatial Index. The R-tree implementation in `choreo-spatial` supports temporal parameterization (time-varying bounding volumes), copy-on-write versioning for snapshot/rollback during verification, and non-AABB predicate dispatch for gaze-cone and proximity queries via GJK/EPA [**?**, **?**].

Verification Backend. The model checker in `choreo-cegar` supports three backends: BDD-based symbolic exploration, SAT-based bounded model checking (BMC), and explicit-state enumeration. Geometric consistency constraints (Theorem 6) are encoded as BDD constraints or SAT clauses to prune infeasible states before exploration.

Code Generation. Generated code targets standalone Rust (`no_std`-compatible state machines), C# (`MonoBehaviour` for Unity/MRTK integration), TypeScript (WebXR session handlers), Graphviz DOT (visualization), and JSON (machine-readable interchange).

9 Evaluation

10 Evaluation

This revision treats evaluation as an artifact-validation section rather than a comparative empirical study. The repository contains an automated benchmark harness over synthetic XR scenarios, JSON scenario descriptions for small-to-stress configurations, and micro-benchmarks for geometric kernels and verification bookkeeping. These artifacts show that the implementation can compile representative choreographies, run the verification pipeline, and emit machine-readable statistics.

10.1 Automated Scenario Harness

The benchmark directory includes four synthetic scenarios of increasing size together with a baseline-results file. They are useful for regression testing the compiler, spatial reasoning code, and verifier on controlled inputs. Because the scenarios are synthetic and the measurements are automated, we do not interpret them as evidence about real deployed XR applications, user behavior, or hardware-specific performance.

10.2 What the Repository Measurements Support

The artifact supports several modest claims. First, the end-to-end pipeline can compile and verify choreographies without XR hardware. Second, the benchmark harness can record states explored, pruning behavior, refinement iterations, and memory usage for synthetic scenarios. Third, the crate-level micro-benchmarks provide repeatable measurements for core geometric primitives such as GJK, BVH queries, and symbolic bookkeeping.

10.3 Removed Claims

We intentionally omit comparative accuracy numbers against production frameworks, manual-testing multipliers, gesture-dataset precision and recall tables, coverage percentages derived from placeholder plots, and build-time comparisons against external toolchains. The current paper therefore makes no user-study claim, no deployment claim, and no claim that repository benchmark scripts establish defect counts in production XR software.

11 Threats to Validity

External Validity. The benchmark suite is synthetic and protocol-oriented. It is designed to exercise the language, compiler, and verifier on representative XR interaction patterns, but it is not extracted from deployed applications and does not include human-subject evaluation.

Internal Validity. The repository includes benchmark scripts, JSON scenarios, and baseline result files, so the claims retained in this paper are limited to what those automated artifacts directly support. We do not rely on hidden logs, unpublished issue triage, or undocumented fallback behavior to substantiate empirical claims.

Construct Validity. The evaluation concerns protocol specification and static verification, not developer usability, runtime UX quality, or perceptual comfort. Those questions require separate studies with real hardware and human participants.

12 Conclusion and Future Work

13 Conclusion and Future Work

We presented Choreo, a language and verification pipeline for XR interaction choreographies. The core contributions that remain fully supported in this revision are the declarative specification language, the decidable spatial type discipline, the spatial CEGAR verification workflow, and the multi-crate Rust implementation that connects parsing, spatial reasoning, automata construction, and code generation.

The repository also contains automated synthetic benchmarks that are useful for regression testing and engineering validation. We treat those artifacts as evidence that the implementation is executable and reproducible, not as a substitute for user studies or deployment-scale comparative evaluation. Future work includes mechanized proofs, extraction from real XR codebases, and empirical studies with actual hardware and developers.

Future Work. We identify four directions for extending Choreo:

- *Lean 4 mechanization*: formalizing the spatial type soundness and CEGAR termination arguments in a proof assistant.

- *Code extraction*: translating larger fragments of existing Unity, WebXR, and related code into the choreography language.
- *Runtime integration*: connecting static certificates with runtime monitors for mixed static/dynamic assurance.
- *Human-centered evaluation*: studying developer usability, learnability, and the practical value of verified interaction specifications on real XR hardware.

References

[1] Allen, J.F.: Maintaining knowledge about temporal intervals. Commun. ACM **26**(11), 832–843 (1983)

[2] Alur, R., Dill, D.L.: A theory of timed automata. Theoret. Comput. Sci. **126**(2), 183–235 (1994)

[3] Ball, T., Rajamani, S.K.: Automatically validating temporal safety properties of interfaces. In: SPIN. pp. 103–122. Springer (2001)

[4] Ball, T., Rajamani, S.K.: The SLAM project: debugging system software via static analysis. In: POPL. pp. 1–3. ACM (2002)

[5] Bocchi, L., Chen, T.-C., Demangeon, R., Honda, K., Yoshida, N.: Monitoring networks through multiparty session types. Theoret. Comput. Sci. **669**, 33–58 (2017)

[6] Calcagno, C., Cardelli, L., Gordon, A.D.: Deciding validity in a spatial logic for trees. J. Funct. Program. **15**(4), 543–572 (2005)

[7] Cardelli, L., Gordon, A.D.: Anytime, anywhere: modal logics for mobile ambients. In: POPL. pp. 365–377. ACM (2001)

[8] Clarke, E., Grumberg, O., Jha, S., Lu, Y., Veith, H.: Counterexample-guided abstraction refinement. In: CAV. LNCS, vol. 1855, pp. 154–169. Springer (2000)

[9] Clarke, E., Grumberg, O., Jha, S., Lu, Y., Veith, H.: Counterexample-guided abstraction refinement for symbolic model checking. J. ACM **50**(5), 752–794 (2003)

[10] Desai, A., Gupta, V., Jackson, E., Qadeer, S., Rajamani, S., Zufferey, D.: P: safe asynchronous event-driven programming. In: PLDI. pp. 321–332. ACM (2013)

[11] van den Bergen, G.: Proximity queries and penetration depth computation on 3D game objects. In: Game Developers Conf. (2001)

[12] Bauer, S., Fahrenberg, U., Legay, A., Thrane, C.: General quantitative specification theories with modalities. In: CSR. LNCS, vol. 7353, pp. 18–30. Springer (2012)

[13] Carbone, P., Katsifodimos, A., Ewen, S., Markl, V., Haridi, S., Tzoumas, K.: Apache Flink: stream and batch processing in a single engine. Bull. IEEE TCDE **36**(4) (2015)

[14] Fremont, D.J., Dreossi, T., Ghosh, S., Yue, X., Sangiovanni-Vincentelli, A.L., Seshia, S.A.: Scenic: a language for scenario specification and scene generation. In: PLDI. pp. 63–78. ACM (2019)

[15] Giallorenzo, S., Montesi, F., Peressotti, M.: Choral: object-oriented choreographic programming. ACM Trans. Program. Lang. Syst. **46**(1), 1–59 (2024)

[16] Gilbert, E.G., Johnson, D.W., Keerthi, S.S.: A fast procedure for computing the distance between complex objects in three-dimensional space. IEEE J. Robot. Autom. **4**(2), 193–203 (1988)

[17] Khronos Group: glTF 2.0 specification (2017). `https://www.khronos.org/gltf/`

[18] Graf, S., Saïdi, H.: Construction of abstract state graphs with PVS. In: CAV. LNCS, vol. 1254, pp. 72–83. Springer (1997)

[19] Guttman, A.: R-trees: a dynamic index structure for spatial searching. In: SIGMOD. pp. 47–57. ACM (1984)

[20] Henzinger, T.A., Jhala, R., Majumdar, R., Sutre, G.: Lazy abstraction. In: POPL. pp. 58–70. ACM (2002)

[21] Hirsch, A.K., Garg, D.: Pirouette: higher-order typed functional choreographies. Proc. ACM Program. Lang. **6**(POPL), 1–27 (2022)

[22] Honda, K., Yoshida, N., Carbone, M.: Multiparty asynchronous session types. In: POPL. pp. 273–284. ACM (2008)

[23] Honda, K., Yoshida, N., Carbone, M.: Multiparty asynchronous session types. J. ACM **63**(1), 1–67 (2016)

[24] Koymans, R.: Specifying real-time properties with metric temporal logic. Real-Time Syst. **2**(4), 255–299 (1990)

[25] McMillan, K.L.: Interpolation and SAT-based model checking. In: CAV. LNCS, vol. 2725, pp. 1–13. Springer (2003)

[26] McMillan, K.L.: Lazy abstraction with interpolants. In: CAV. LNCS, vol. 4144, pp. 123–136. Springer (2006)

[27] Meta: Interaction SDK documentation (2023). `https://developer.oculus.com/documentation/unity/unity-isdk-interaction-sdk-overview/`

[28] Montesi, F., Yoshida, N.: Compositional choreographies. In: CONCUR. LNCS, vol. 8052, pp. 425–439. Springer (2013)

[29] Microsoft: Mixed Reality Toolkit (MRTK) v2.8 (2023). `https://github.com/microsoft/MixedRealityToolkit-Unity`

[30] Mueller, E.T.: Commonsense Reasoning: An Event Calculus Based Approach. Morgan Kaufmann (2006)

[31] O'Hearn, P.W.: Separation logic. Commun. ACM **62**(2), 86–95 (2019)

[32] Khronos Group: OpenXR specification (2019). `https://www.khronos.org/openxr/`

[33] Prasetya, I.S.W.B., et al.: iv4XR: a research framework for testing extended reality systems. SoftwareX **18**, 101096 (2022)

[34] Apple: RealityKit documentation (2023). `https://developer.apple.com/documentation/realitykit`

[35] Reynolds, J.C.: Separation logic: a logic for shared mutable data structures. In: LICS. pp. 55–74. IEEE (2002)

[36] Shanahan, M.: The event calculus explained. In: Artif. Intell. Today. LNCS, vol. 1600, pp. 409–430. Springer (1999)

[37] Shen, G., Kashiwa, S., Kuper, L.: HasChor: functional choreographic programming for all. Proc. ACM Program. Lang. **7**(ICFP), 207 (2023)

[38] Behrmann, G., David, A., Larsen, K.G.: A tutorial on Uppaal. In: SFM. LNCS, vol. 3185, pp. 200–236. Springer (2004)

[39] Pixar: Universal Scene Description (USD) specification (2019). `https://openusd.org/`

[40] W3C: WebXR Device API (2023). `https://www.w3.org/TR/webxr/`

[41] Yoshida, N., Hu, R., Neykova, R., Ng, N.: The Scribble protocol language. In: TGC. LNCS, vol. 8358, pp. 22–41. Springer (2014)

[42] Zhou, F., Ferreira, F., Yoshida, N.: Multiparty session type refinements with dependent types (2020)

[43] Unity Technologies: XR Interaction Toolkit v2.5 (2023). `https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@2.5/`

[44] Epic Games: Unreal Engine XR Development (2023). `https://docs.unrealengine.com/5.3/en-US/developing-for-xr-experiences-in-unreal-engine/`

[45] A-Frame: Web framework for building VR experiences (2023). `https://aframe.io/`

[46] Poimandres: React XR – React Three Fiber XR integration (2023). `https://github.com/pmndrs/xr`

[47] Valve: SteamVR Input System (2023). `https://partner.steamgames.com/doc/features/steamvr/input`

[48] Khronos Group: OpenXR Conformance Test Suite (2023). `https://github.com/KhronosGroup/OpenXR-CTS`

[49] W3C Immersive Web Working Group: Web Platform Tests – WebXR (2023). `https://wpt.fyi/results/webxr`

[50] De Smedt, Q., Wannous, H., Vandeborre, J.-P., Guerry, J., Le Saux, B., Filliat, D.: SHREC'17 track: 3D hand gesture recognition using a depth and skeletal dataset. In: Eurographics Workshop on 3D Object Retrieval. pp. 33–38 (2017)

[51] Zhang, Y., Cao, C., Cheng, J., Lu, H.: EgoGesture: a new dataset and benchmark for egocentric hand gesture recognition. IEEE Trans. Multimedia **20**(5), 1038–1050 (2018)

[52] LaViola, J.J., Kruijff, E., McMahan, R.P., Bowman, D.A., Poupyrev, I.: 3D User Interfaces: Theory and Practice, 2nd edn. Addison-Wesley (2017)